



DEPARTMENT OF ELECTRICAL ENGINEERING AND INFORMATION TECHNOLOGY
POLYTECHNIC SCHOOL OF BASIC SCIENCES

MASTER'S IN AUTOMATION AND ROBOTICS ENGINEERING

CAR-LIKE ROBOT: PLANNING, CONTROL & LOCALIZATION

Professor:
Prof. Fabio Ruggiero

Candidates:
Giuseppe Arena [P38000311]
Nicholas Ruggiero [P38000312]

Academic Year 2025/2026

1 Introduction

2 Kinematic Model

3 Planning

- Map Processing
- Graph Generation via Reeds-Shepp
- Optimal Path Search via Dijkstra
- Trajectory Generation and Velocity Profiles
- Uniform Scaling for Physical Constraints

4 Control

- I/O Feedback Linearization
- Ideal Feedback Results

5 Localization

- Odometry
- Euler Odometry: Implementation
- Forward Euler Odometry and Results
- RK2 Odometry: Implementation
- Runge-Kutta 2 (RK2) Odometry and Results
- Active Localization with Extended Kalman Filter
- Exteroceptive Sensing: LiDAR
- Active Localization Results

6 Conclusion

The autonomous navigation of non-holonomic vehicles requires a robust architecture integrating motion planning, smooth trajectory generation, and stable control.

The **main goal** is design a complete guidance and control framework for a car-like robot in a mapped workspace.

The vehicle must be driven from a start to a goal configuration, avoiding obstacles and respecting mechanical constraints (minimum turning radius, maximum velocity limits).

Methodology stages:

- Geometric path synthesis via Reeds-Shepp curves.
- Time parameterization and reference trajectory generation via Dijkstra.
- Trajectory tracking with pure Odometry methods (Euler & RK2).
- Trajectory Tracking through active localization.

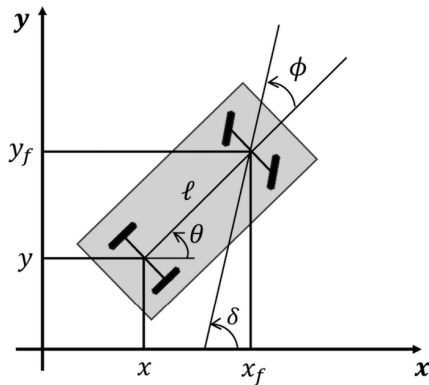


Figure: Car-like kinematic model.

- Ackermann Kinematic Model implemented.
- **Pure rolling constraints**, one for each wheel, preventing lateral slip.
- State vector $\mathbf{q} = [x, y, \theta, \phi]^T$.

The forward kinematics mapping control inputs $\mathbf{u} = [v, \omega]^T$ to state derivatives is:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \\ \dot{\phi} \end{bmatrix} = \begin{bmatrix} \cos \theta \\ \sin \theta \\ \frac{\tan \phi}{l} \\ 0 \end{bmatrix} v + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \omega \quad (1)$$

Map Processing

The environment is initially provided as a real-world map extraction then **the map is binarized** for path-finding algorithms.

- **Roads** are assigned a logical value of 1.
- **Obstacles** and unnavigable areas are assigned 0.

The resulting matrix is transposed to properly align with the Cartesian (x, y) coordinate system. This matrix acts as the absolute collision-checking reference for all subsequent path expansions.

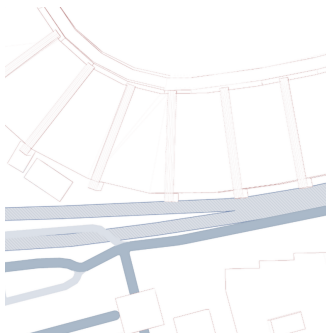


Figure: Original map.



Figure: Binarized map.

A customized reachability graph is built through iterative forward-simulation of **Reeds-Shepp** motion primitives.

Starting from the initial pose, the robot's future states are explored by applying:

- **Three control actions:** straight forward, turn left, turn right.
- **Constraints:** Fixed straight length and fixed angular variation for curves ($\pi/2$ rad) using minimum turning radius R_{min} .
- **Collision checking:** every generated primitive is densely discretized into intermediate waypoints against the occupancy grid.

This iterative breadth-first expansion continues until a tree of kinematically executable and safe routes densely cover the space between start and end position selected by user.

Optimal Path Search via Dijkstra

Once the graph of feasible Reeds-Shepp primitives is fully generated, the spatial relationships between the valid nodes are formalized into an **adjacency matrix**, then the desired optimal path is generated following these steps:

- The algorithm calculates the **Euclidean distance** from all generated nodes to the target.
- The **node closest to the target is selected** as the designated arrival point.
- **Dijkstra's algorithm** finds the optimal sequence of primitives, yielding a collision-free geometric route.

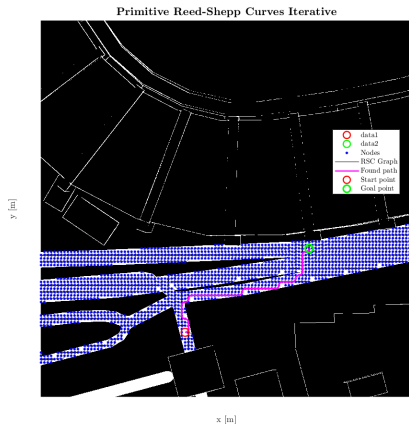


Figure: Generated Reeds-Shepp graph and optimal path via Dijkstra.

To allow a feedback controller to track the path, it must be parameterized in time, defining exactly when the robot should be at a specific coordinate.

Based on the k road curvature:

$$\kappa = \frac{|x'y'' - y'x''|}{(x'^2 + y'^2)^{1.5}} \quad (2)$$

the target velocity profile is systematically generated:

- Maximum velocity (2.5 m/s) on straights
- Reduced safety velocity (0.6 m/s) on high-curvature regions ($\kappa > 0.04$).
- A Gaussian moving average filter and raised-cosine window at the trajectory extremis ensure continuous acceleration ramps, preventing jerks.

Finally, the spatial profile is mathematically transformed into the time domain via numerical integration ($dt = ds/v$) and the entire trajectory is obtained.

Uniform Scaling for Physical Constraints

To prevent actuator saturation, a Uniform Scaling step is enforced, respecting hardware limits imposed ($v_{max} = 3.0$ m/s, $\omega_{max} = 2.0$ rad/s):

$$k = \max \left(1, \frac{v_{peak}}{v_{max}}, \frac{\omega_{peak}}{\omega_{max}} \right) \quad (3)$$

If $k > 1$, the total execution time is uniformly stretched ($T_{new} = k \cdot T$), strictly guaranteeing kinematic safety while preserving the geometric path.

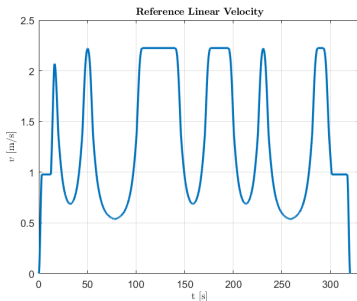


Figure: Reference linear velocity $v(t)$.

I/O Feedback Linearization

Standard feedback directly on the center of mass fails due to non-holonomic singular decoupling matrices, so a **I/O Feedback Linearization controller** is implemented.

Thanks to it, the system is mathematically transformed into a linear, decoupled system, allowing the application of standard linear control techniques.

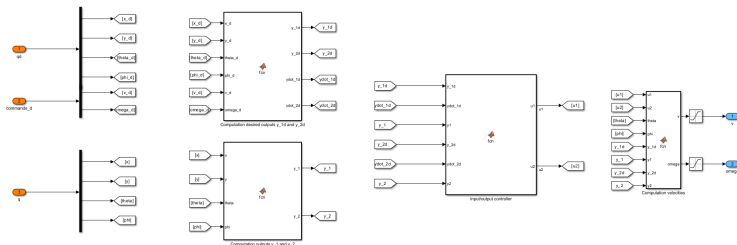


Figure: IO Feedback Linearization Controller Scheme.

The controller is divided into four main functional blocks, each responsible for a specific step in the linearization and tracking process.

- **Computation of Outputs:** Desired (y_{1d}, y_{2d}) and actual (y_1, y_2) coordinates of the virtual point are calculated using look-ahead distance Δ .

$$\begin{cases} y_{1d} = x_d + l \cos(\theta_d) + \Delta \cos(\theta_d + \phi_d) \\ y_{2d} = y_d + l \sin(\theta_d) + \Delta \sin(\theta_d + \phi_d) \end{cases} \quad \begin{cases} y_1 = x + l \cos(\theta) + \Delta \cos(\theta + \phi) \\ y_2 = y + l \sin(\theta) + \Delta \sin(\theta + \phi) \end{cases} \quad (4)$$

- **I/O Controller:** Proportional tracking law computes auxiliary inputs (u_1, u_2) .

$$\begin{cases} u_1 = \dot{y}_{1d} + k_1(y_{1d} - y_1) \\ u_2 = \dot{y}_{2d} + k_2(y_{2d} - y_2) \end{cases} \quad (5)$$

These auxiliary inputs represent the required Cartesian velocities of the virtual point to aggressively eliminate the tracking error.

- **Computation of Physical Velocities:** Inverse kinematic matrix maps auxiliary inputs to physical variables (v_{raw}, ω_{raw}) .

$$\begin{bmatrix} v_{raw} \\ \omega_{raw} \end{bmatrix} = T^{-1}(\theta, \phi) \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} \quad (6)$$

Ideal Feedback Results

An initial simulation was conducted under nominal conditions. In this scenario, the controller relies on **ideal feedback**, meaning it receives the exact true state of the robot without any sensor noise, or odometric/localization errors.

- Actual trajectory (blue) perfectly overlays the desired reference path (green dashed).
- State errors remain very low and bounded.
- Bounded and clean control commands demonstrate theoretical correctness.

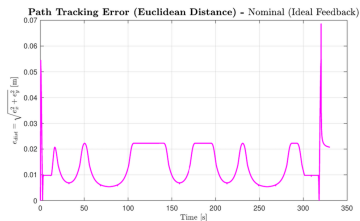
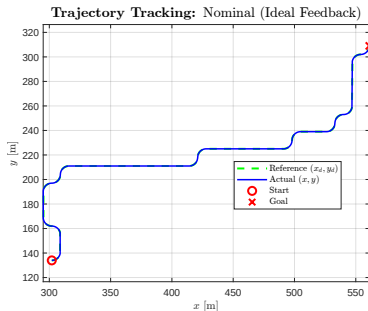


Figure: Trajectory Tracking & Tracking Error

Ideal Feedback: Errors and Controls

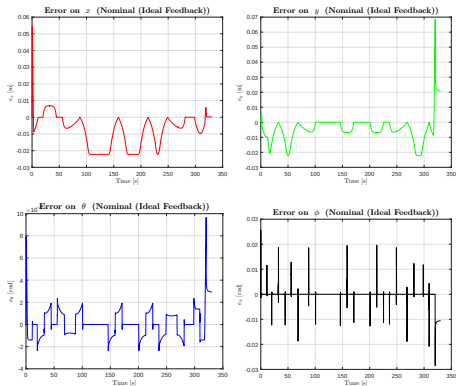
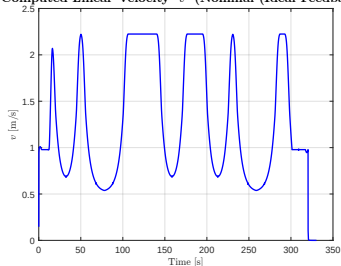


Figure: Tracking Errors

Computed Linear Velocity v (Nominal (Ideal Feedback))



Computed Steering Angular Velocity ω (Nominal (Ideal Feedback))

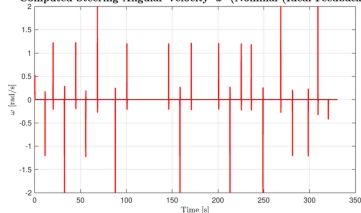


Figure: Control Inputs

In real-world applications, the true state of the robot is not directly accessible. Instead, the vehicle must estimate its current pose and configuration (x, y, θ, ϕ) using onboard sensory data. This localization process is essential for providing the feedback controller with the necessary information to compute the corrective tracking actions.

State estimation has been realized following two different approaches: **Odometry & Active Localization**.

Odometry relies on integrating the measured control velocities over time to estimate the robot's displacement from a known starting position. For it two integration methods evaluated:

- **Forward Euler Method** (First-order).
- **Runge-Kutta 2** (Second-order midpoint approximation).

- **Method:** First-order numerical integration.
- **Logic:** Computes the next state based strictly on the derivatives at the current time step k .

Discrete Update Equations

$$\begin{cases} x_{k+1} = x_k + v_{mis} T_s \cos(\theta_k) \\ y_{k+1} = y_k + v_{mis} T_s \sin(\theta_k) \\ \theta_{k+1} = \theta_k + \frac{v_{mis}}{l} \tan(\phi_k) T_s \\ \phi_{k+1} = \phi_k + \omega_{mis} T_s \end{cases} \quad (7)$$

- **Pros:** Extremely low computational cost.
- **Cons:** **Truncation Error.** Assumes constant rates over T_s , leading to rapid, unbounded numerical drift in non-linear paths.

Forward Euler Odometry and Results

- Computes next state strictly based on current derivatives.
- Constant derivative assumption introduces severe truncation error in curves.
- **Results:** Actual trajectory significantly diverges.
- Unbounded error grows continuously, reaching over 60 meters.

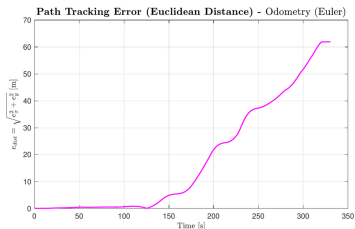
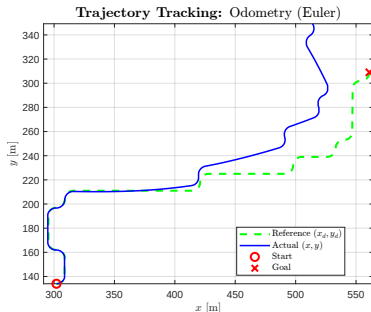


Figure: Trajectory Tracking & Tracking Error

Forward Euler: Errors and Controls

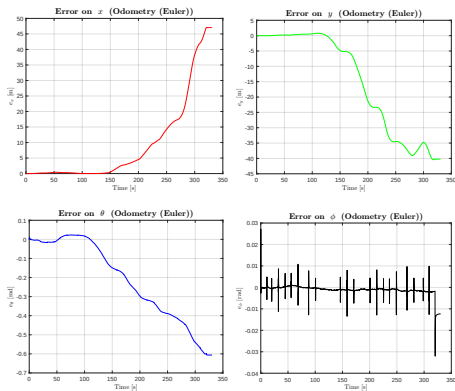


Figure: Tracking Errors

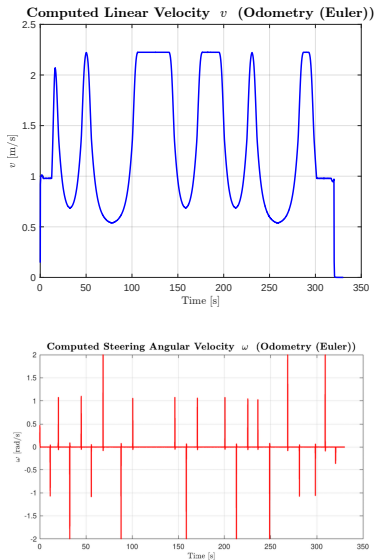


Figure: Control Inputs

- **Method:** Second-order numerical integration.
- **Step 1: Midpoint Prediction**

$$\phi_m = \phi_k + 0.5\omega_{mis} T_s, \quad \theta_m = \theta_k + 0.5 \frac{v_{mis}}{l} \tan(\phi_k) T_s \quad (8)$$

- **Step 2: State Integration**

$$x_{k+1} = x_k + v_{mis} T_s \cos(\theta_m), \quad y_{k+1} = y_k + v_{mis} T_s \sin(\theta_m) \quad (9)$$

- **Pros:** Mathematically superior local estimate (reduces local truncation error).
- **Cons:** **Open-loop nature.** Without external corrections, even small integration errors accumulate over time, failing long-term tracking.

Runge-Kutta 2 (RK2) Odometry and Results

- Evaluates kinematic rates at the midpoint of the interval.
- Mathematically superior local estimate reduces truncation errors.
- **Results:** Due to the lack of absolute correction, small integration errors still accumulate.
- Tracking error still unbounded, robot drifts completely off track.

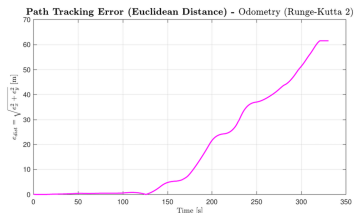
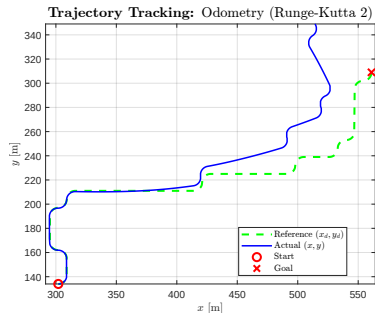


Figure: Trajectory Tracking & Tracking Error

Runge-Kutta 2: Errors and Controls

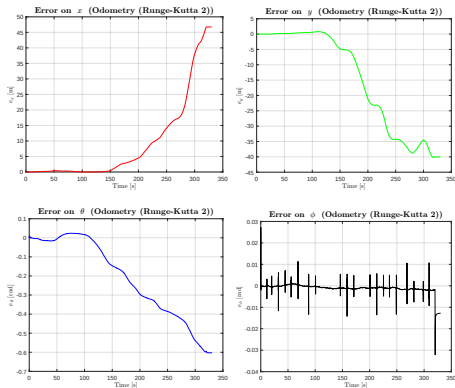
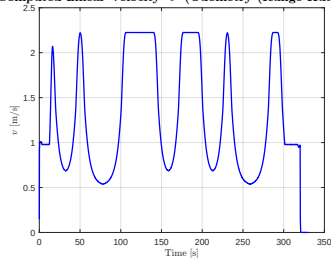


Figure: Tracking Errors

Computed Linear Velocity v (Odometry (Runge-Kutta 2))



Computed Steering Angular Velocity ω (Odometry (Runge-Kutta 2))

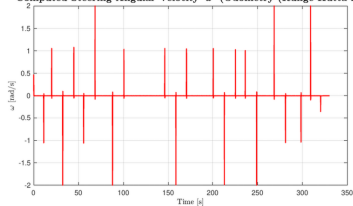


Figure: Control Inputs

- Solves unbounded error accumulation through closed-loop Active Localization.
- **Extended Kalman Filter (EKF):** Fuses proprioceptive (odometry) and exteroceptive (sensor) data.
- **Prediction Step:** Propagates state and covariance utilizing RK2 for superior local approximation.
- **Update Step:** Formal correction utilizing external measurements and Kalman Gain weighting.

- **Prediction (Proprioceptive):** Uses RK2 for state propagation and updates the covariance P :

$$P_{pred} = F P_{prev} F^T + W Q_{ekf} W^T \quad (10)$$

- **Update (Exteroceptive):** Computes Kalman Gain K and corrects the estimate based on Innovation ($z_{meas} - \hat{z}$):

$$K = P_{pred} H^T (H P_{pred} H^T + R_{ekf})^{-1} \quad (11)$$

$$\hat{q}_{new} = \hat{q}_{pred} + K(z_{meas} - \hat{z}) \quad (12)$$

- **Result:** Successfully **bounds the drift**, although it introduces high-frequency chattering in control signals.

- **Why LiDAR?** Chosen as the industry standard for autonomous vehicles due to high accuracy in range and bearing detections.
- **Landmark Setup:**
 - 8 structural landmarks systematically extracted during the planning phase.
 - Positioned at a 15-meter orthogonal offset from the reference trajectory.
- **Objective:** Provide exteroceptive (external) measurements to the EKF to correct the unbounded growth of odometric error.

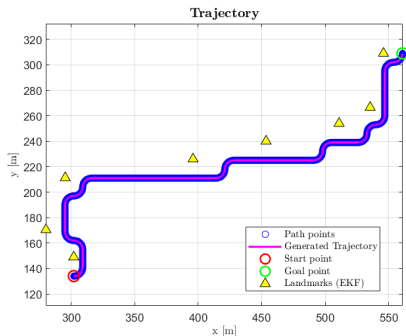


Figure: Environment setup with distributed landmarks.

- **Measurement Vector:** $z_{meas} = [\rho, \alpha]^T$

Observation Equations (Non-linear)

For each visible landmark i at (x_i, y_i) :

$$\begin{cases} \rho_i = \sqrt{(x_i - x)^2 + (y_i - y)^2} \\ \alpha_i = \text{atan2}(y_i - y, x_i - x) - \theta \end{cases} \quad (13)$$

- **Sensor Uncertainty:** Gaussian noise injected via covariance matrix R_{ekf} .

- The EKF effectively bounds the drift.
- Robot accurately reaches the goal point respecting geometric constraints.
- Euclidean error remains strictly bounded.
- Some high-frequency chattering in control signals is present due to stochastic LiDAR updates, but stays within safe limits.

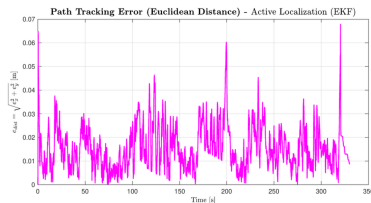
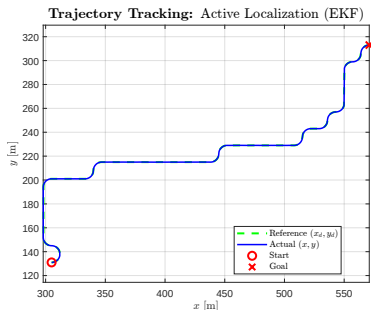


Figure: Trajectory Tracking & Tracking Error

EKF Localization: Errors and Controls

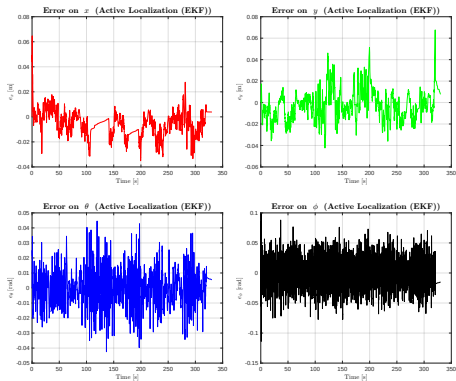
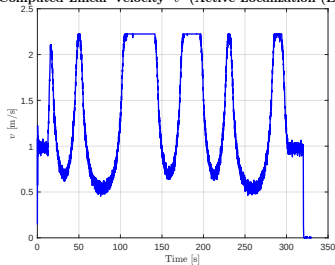


Figure: Tracking Errors

Computed Linear Velocity v (Active Localization (EKF))



Computed Steering Angular Velocity ω (Active Localization (EKF))

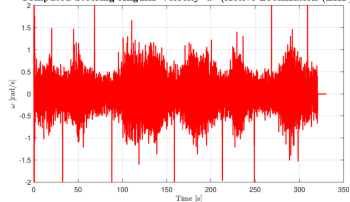


Figure: Control Inputs

- I/O Feedback Linearization successfully overcomes non-holonomic constraints by transforming into a linear system.
- Under ideal feedback, the controller achieves near-perfect tracking, validating proportional gains.
- **Primary limitation:** Extreme vulnerability to state estimation errors; degradation rapidly destabilizes the tracking path.

- **Ideal Feedback:** Flawless but unrealistic for physical systems.
- **Euler Odometry:** Severe drift due to first-order truncation; total tracking breakdown.
- **RK2 Odometry:** Superior local integration, but remains open-loop. Fails long-term.
- **Active Localization (EKF):** Successfully bounds drift by fusing RK2 prediction with LiDAR measurements. Represents the absolute necessity for real-world application, despite introducing minor high-frequency chattering.

Thank You for Your Attention